

**UNIVERSIDADE FEDERAL DE ITAJUBÁ
CAMPUS ITABIRA
INSTITUTO DE CIÊNCIAS TECNOLÓGICAS
ENGENHARIA DE COMPUTAÇÃO**

**STUDY FLOW: SISTEMA DE GERENCIAMENTO
DE ESTUDOS E PRODUTIVIDADE**

Relatório Técnico Final da Disciplina
ECOi2215 – Análise e Desenvolvimento de Software III

INTEGRANTES DO GRUPO 8:

Beatriz Coelho Pereira (Matrícula: d2022010992)
Breno Vítor de Paula (Matrícula: d2022012012)
Carlos Eduardo Abreu da Silva (Matrícula: d2022004556)
Lucas Luz Souza Pires (Matrícula: d2022003076)
Vítor Hugo Rodrigues Basílio (Matrícula: d2023011321)

PROFESSORES COORDENADORES:

Eduardo Ribeiro Felipe
Fabiana Costa Guedes
Rossana de Paula Junqueira Almeida
Wendell Fioravante da Silva Diniz

ITABIRA - MG

2026

Sumário

1	Introdução	2
2	Desenvolvimento de Frontend	2
2.1	Frameworks e Tecnologias	2
2.2	Linguagem de Design	3
2.3	Evolução do Frontend no Módulo III	3
3	Desenvolvimento de Backend	4
3.1	Arquitetura e Serviços Utilizados	4
3.2	Padrões de Projeto Aplicados	4
4	Desenvolvimento da Camada de Persistência	4
4.1	Esquema de Tabelas (Dicionário do Banco)	5
4.2	Gatilhos de Criação Automática de Perfil	5
4.3	Políticas de Segurança em Nível de Linha (RLS)	6
4.4	Otimização e Índices	6
5	Conclusão e Próximos Passos	7
5.1	O que funcionou	7
5.2	Limitações e Backlog Técnico	7

1 Introdução

O projeto **Study Flow** consiste em uma aplicação voltada para a produtividade de estudantes, com foco na organização acadêmica, automação de rotinas de planejamento e gerenciamento do tempo de estudo. A proposta principal do sistema busca integrar compromissos recebidos por e-mail de plataformas acadêmicas de forma a automatizar a geração e distribuição de atividades no cronograma. Com isso, busca-se mitigar a sobrecarga mental dos alunos no planejamento diário, permitindo que dediquem mais foco ao aprendizado efetivo.

Ao final do Módulo II da disciplina de Análise e Desenvolvimento de Software III, o projeto contava com os seguintes artefatos desenvolvidos:

- **Protótipo de Média/Alta Fidelidade no Figma:** Definição completa do fluxo do usuário e do design system, englobando telas de Login, Cadastro, Dashboard, Calendário, Atividades, Disciplinas, Temporizador, Estatísticas, Conquistas/Recompensas e Configurações.
- **Interface Frontend Inicial em Next.js:** Implementação inicial das rotas do frontend do aplicativo.
- **Simulação de Estado via Armazenamento Local (localStorage):** Devido à ausência de um backend real na fase anterior, as lógicas de autenticação, cadastro de contas, tarefas do dashboard, adição de disciplinas e notas eram simuladas no lado do cliente através de persistência em `localStorage` no navegador.

O objetivo principal deste Módulo III foi substituir a simulação local por uma integração real com uma infraestrutura de backend e persistência de dados em nuvem, garantindo a integridade dos dados e a segurança das credenciais.

2 Desenvolvimento de Frontend

A interface gráfica do Study Flow foi construída seguindo padrões modernos de desenvolvimento web baseados em componentes reaproveitáveis, com forte atenção à responsividade e experiência do usuário (UX).

2.1 Frameworks e Tecnologias

O ecossistema do frontend do Study Flow compreende as seguintes tecnologias:

- **Next.js (v16.2.x):** Framework React focado em desempenho, utilizando roteamento dinâmico baseado em arquivos na estrutura `src/app/`. O compilador Turbopack foi utilizado no ambiente de desenvolvimento para carregamento ágil das alterações de estilo e estrutura.
- **React (v19.x):** Biblioteca base para construção das interfaces com o uso extensivo de Hooks (`useState`, `useEffect`, `useCallback`, `useRef`) para controle de estado local e ciclos de vida.
- **Tailwind CSS (v4):** Framework utilitário de CSS que otimiza a estilização através de classes declarativas diretamente nos arquivos TypeScript XML (TSX), assegurando layouts responsivos rápidos e estilização consistente sem acúmulo de CSS redundante.

- **Lucide React:** Conjunto de ícones vetoriais modernos e leves utilizados para representar as ações da barra lateral (*sidebar*), botões e cabeçalhos.
- **TypeScript:** Superset do JavaScript que adiciona tipagem estática ao código, aumentando a manutenibilidade, autocomplete do editor e prevenindo erros de tipo em tempo de desenvolvimento.

2.2 Linguagem de Design

A identidade visual e a linguagem de design adotadas foram projetadas para induzir a sensação de foco, calma e organização:

- **Paleta de Cores:** Composta por tons pastéis baseados em verde-oliva escuro (#29645e), verde-menta claro (#f7fdfd), cinza neutro e detalhes em dourado/amarelo para elementos de gamificação (representando conquistas e pontuações).
- **Tipografia:** Utilização de fontes modernas do Google Fonts, especificamente a família de fontes sem serifa *Roboto* para textos comuns de leitura e *Rubik Bubbles* para logotipos e cabeçalhos decorativos.
- **Responsividade e Animações:** A barra lateral é fixa à esquerda em telas grandes e adaptável/ocultável em dispositivos móveis. Foram criadas micro-animações como o anel de progresso SVG dinâmico na tela do Temporizador Pomodoro, que exhibe visualmente o tempo restante e altera o preenchimento de forma circular suave.

2.3 Evolução do Frontend no Módulo III

A principal alteração aplicada no frontend foi a substituição de funções puras do `localStorage` por chamadas de API assíncronas assinaladas ao SDK do Supabase (`@supabase/supabase-js`). O fluxo de login e cadastro foi reestruturado para verificar primeiro a autenticação junto à nuvem e, em caso de sucesso, persistir os dados da sessão localmente para evitar acessos não autorizados.

Para estender a flexibilidade da ferramenta e atender à especificação de autenticação ágil, foi criada uma função RPC (Remote Procedure Call) no banco de dados chamada `get_email_by_username`. Essa função foi integrada ao fluxo de login do frontend para permitir que o usuário faça login inserindo tanto seu e-mail de cadastro quanto seu nome de usuário único (*username*), como pode ser observado no método `resolveLoginEmail` no código-fonte principal de login:

```

1 const resolveLoginEmail = useCallback(async () => {
2   const login = email.trim();
3   if (login.includes("@")) {
4     return login;
5   }
6   const { data } = await supabase.rpc("get_email_by_username", {
7     login_username: login,
8   });
9   if (typeof data === "string" && data.includes("@")) {
10    return data;
11  }
12  return login;
13 }, [email]);

```

Listing 1: Resolução de login por usuário ou e-mail no frontend

Se a conexão com a nuvem falhar ou não existirem chaves de configuração declaradas nas variáveis de ambiente (`.env`), o sistema executa um fluxo de contingência (*fallback*) para o modo demonstração (*Demo Mode*), que lê os usuários cadastrados e sessões de estudo armazenadas no `localStorage`, garantindo que a aplicação possa ser demonstrada mesmo offline.

3 Desenvolvimento de Backend

O Study Flow adotou um modelo arquitetural baseado em **Backend-as-a-Service (BaaS)**, utilizando o ecossistema **Supabase**. Essa decisão permitiu acelerar o desenvolvimento ao fornecer de forma integrada serviços essenciais de infraestrutura.

3.1 Arquitetura e Serviços Utilizados

O backend em nuvem é composto por três blocos principais de serviços:

1. **Supabase Auth**: Serviço de gerenciamento de identidades e acessos. Gerencia a criação de contas, redefinição de senhas e a emissão de tokens de sessão seguros baseados em JSON Web Tokens (JWT) passados no cabeçalho das requisições.
2. **Banco de Dados Relacional (PostgreSQL)**: SGBD robusto utilizado para armazenamento de dados estruturados. O Supabase fornece suporte nativo a extensões espaciais e criptográficas (`pgcrypto`).
3. **Row Level Security (RLS)**: Camada de segurança interna do banco que filtra e valida acessos diretamente em nível de linha com base no ID do usuário logado na sessão ativa.

3.2 Padrões de Projeto Aplicados

- **BaaS / Serverless**: Elimina a necessidade de gerenciar servidores de aplicação dedicados, utilizando requisições diretas do cliente para os endpoints do banco e autenticação.
- **Observer Pattern via Database Triggers**: Padrão utilizado para propagar alterações automáticas. Um gatilho foi configurado para observar a tabela de credenciais nativas do Supabase (`auth.users`). Quando um novo usuário se cadastra, o trigger é disparado automaticamente para sincronizar os dados básicos e criar um registro na tabela pública de perfis (`public.profiles`).
- **Stored Procedures / RPC**: Lógicas encapsuladas no banco de dados para evitar exposição de queries complexas no frontend, como a resolução de e-mails via nome de usuário.

4 Desenvolvimento da Camada de Persistência

A camada de persistência de dados foi totalmente redesenhada a partir do modelo físico do banco de dados definido na disciplina anterior (ADS II). O banco de dados relacional foi hospedado no PostgreSQL do Supabase, contendo tabelas bem estruturadas, regras de integridade referencial por chaves estrangeiras e gatilhos automatizados em PL/pgSQL.

4.1 Esquema de Tabelas (Dicionário do Banco)

A estrutura física de dados é definida pelas seguintes tabelas do esquema `public`:

- **profiles**: Extensão da tabela interna do Supabase (`auth.users`). Armazena as informações públicas de perfil do usuário. Chave primária: `id` (UUID, chave estrangeira de `auth.users`). Atributos adicionais: `nome`, `username`, `email`, `avatar_url`, `horas_diarias`, `nivel_atual`, `xp`.
- **disciplinas**: Guarda as matérias acadêmicas criadas pelos estudantes. Atributos: `id` (UUID), `user_id` (referência a `profiles`), `nome`, `professor`, `data_inicio`, `data_fim`, `anotacoes` (tipo `jsonb` para flexibilidade no editor de texto de notas).
- **avaliacoes**: Provas, trabalhos ou outras atividades pontuadas de uma disciplina. Atributos: `id`, `disciplina_id` (chave estrangeira), `nome`, `tipo` (enum: prova, trabalho, outro), `data_entrega`.
- **topicos**: Conteúdos específicos e unidades de estudo atrelados a uma disciplina.
- **metas**: Objetivos de estudos definidos pelo usuário (ex: estudar 5 horas de Algoritmos). Contém campo de controle de status (`pendente`, `em_andamento`, `feito`, `atrasado`).
- **itens_cronograma**: Atividades gerais que aparecem no calendário e na lista de afazeres. Referencia opcionalmente uma `disciplina_id` ou um `topico_id`.
- **sessoes_estudo**: Registros das execuções do temporizador Pomodoro. Salva as datas exatas de início, fim, duração em minutos, número de pausas durante a sessão e a pontuação obtida em XP.
- **conquistas** e **user_conquistas**: Tabelas responsáveis pelo motor de gamificação. A tabela `conquistas` armazena a definição (ex: “Mestre do Tempo”), o critério técnico em formato JSON e o XP de recompensa. A tabela associativa `user_conquistas` registra quais usuários desbloquearam quais conquistas e em que data.
- **palavras_chave** e **sinc_gmail**: Configurações do usuário e palavras-chave de triagem para a sincronização automatizada com e-mails acadêmicos.

4.2 Gatilhos de Criação Automática de Perfil

Para garantir que cada usuário autenticado possua um registro correspondente na tabela `profiles`, foi desenvolvida uma trigger na tabela interna `auth.users` que dispara uma função em PL/pgSQL no momento em que a conta é criada no Supabase Auth:

```
1 CREATE OR REPLACE FUNCTION public.handle_new_user()  
2 RETURNS TRIGGER  
3 LANGUAGE plpgsql  
4 SECURITY DEFINER  
5 SET search_path = public  
6 AS $$  
7 BEGIN  
8     INSERT INTO public.profiles (id, nome, username, email)  
9     VALUES (  
10         new.id,
```

```

11     COALESCE(new.raw_user_meta_data ->> 'nome', new.raw_user_meta_data
12     ->> 'full_name', split_part(new.email, '@', 1)),
13     COALESCE(new.raw_user_meta_data ->> 'username', split_part(new.email
14     , '@', 1)),
15     new.email
16 )
17 ON CONFLICT (id) DO NOTHING;
18 RETURN new;
19 END;
20 $$;
21
22 CREATE TRIGGER on_auth_user_created
23 AFTER INSERT ON auth.users
24 FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

Listing 2: Trigger de criação de perfil de usuário na base de dados

4.3 Políticas de Segurança em Nível de Linha (RLS)

Seguindo os princípios de privacidade do usuário e em conformidade com a LGPD (Lei Geral de Proteção de Dados), todas as tabelas foram configuradas com **Row Level Security (RLS)**. Por padrão, nenhum usuário consegue visualizar ou alterar dados que pertencem a outra pessoa.

As políticas criadas utilizam a função interna do Supabase `auth.uid()` para comparar o ID do token de autenticação ativo com a coluna de relacionamento da tabela correspondente:

```

1  -- Habilita RLS para a tabela disciplinas
2  ALTER TABLE public.disciplinas ENABLE ROW LEVEL SECURITY;
3
4  -- Politica que permite selecao, insercao e atualizacao apenas dos dados
5  -- pertencentes ao proprio usuario
6  CREATE POLICY "disciplinas_all_own" ON public.disciplinas
7  FOR ALL USING (auth.uid() = disciplinas.user_id)
8  WITH CHECK (auth.uid() = disciplinas.user_id);

```

Listing 3: Exemplo de políticas RLS aplicadas à tabela de disciplinas

4.4 Otimização e Índices

Para acelerar o tempo de carregamento de páginas críticas (como calendário e estatísticas), foram criados índices de árvore-B sobre as colunas mais utilizadas em cláusulas de junção (JOIN) e filtragem (WHERE):

- `idx_disciplinas_user` na coluna `user_id` da tabela `disciplinas`.
- `idx_itens_user` na coluna `user_id` da tabela `itens_cronograma`.
- `idx_sesoes_user` na coluna `user_id` da tabela `sesoes_estudo`.
- `idx_disciplinas_anotacoes_gin`: Índice do tipo GIN (Generalized Inverted Index) aplicado à coluna `anotacoes (jsonb)` da tabela `disciplinas`, otimizando buscas por termos textuais dentro do editor de notas.

5 Conclusão e Próximos Passos

A transição do Study Flow de um sistema puramente estático simulado em memória local para uma aplicação web conectada a um serviço de banco de dados e autenticação escalável em nuvem representou um salto de maturidade no projeto.

5.1 O que funcionou

- **Consistência da Stack BaaS:** O Supabase forneceu um fluxo extremamente estável de autenticação e comunicação com o banco através de consultas assíncronas simples em JavaScript.
- **Segurança Automatizada:** As políticas RLS e os gatilhos em nível de banco de dados se mostraram excelentes para garantir que o frontend não precise se preocupar em programar verificações complexas de permissão, uma vez que o próprio banco de dados bloqueia ou sincroniza os dados automaticamente.
- **Resiliência Offline:** A arquitetura híbrida permitiu manter o `localStorage` como mecanismo de contingência, permitindo o uso imediato das ferramentas em modo offline de demonstração.

5.2 Limitações e Backlog Técnico

Apesar dos avanços, algumas funcionalidades do escopo inicial da aplicação permanecem em nível de planejamento ou modelagem lógica de dados no banco, necessitando de refinamento nas próximas etapas de desenvolvimento:

- **Integração de Sincronização de E-mail (Gmail API):** Embora a estrutura de persistência com tabelas para `palavras_chave` e status de sincronização esteja concluída no PostgreSQL, a implementação do fluxo OAuth e leitura da caixa de e-mails em tempo real no frontend foi deixada para o próximo módulo.
- **Edição Direta do Perfil no Banco:** A tela de perfil está funcional localmente para personalizações cosméticas, mas o espelhamento das modificações nos registros em nuvem do Supabase precisa de ajustes na integração das APIs.
- **Estatísticas e Gráficos de Longo Prazo:** A exibição de dados está vinculada à duração em minutos das sessões do Pomodoro. O motor de cálculo estatístico semanal e mensal precisará ser expandido para agregar tarefas concluídas fora do temporizador de foco.

O desenvolvimento atual consolida uma base sólida de engenharia de software, aderindo às boas práticas de arquitetura web, segurança de dados e separação de preocupações entre interface e lógica de persistência.